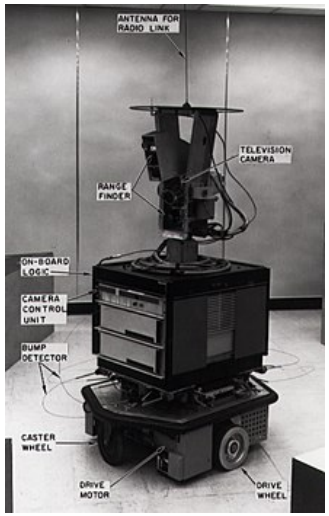
AI and Robotics

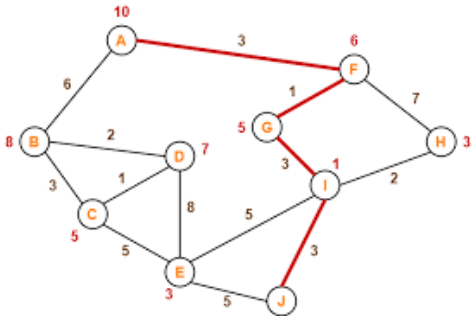
Lesson 2a: From high-level commands to motion: introduction to motion planning and control

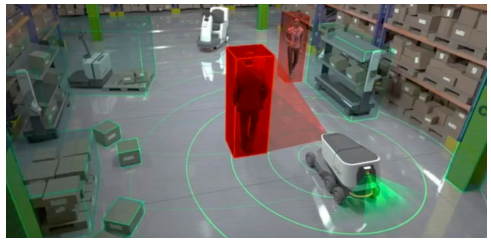
Sarah Keren

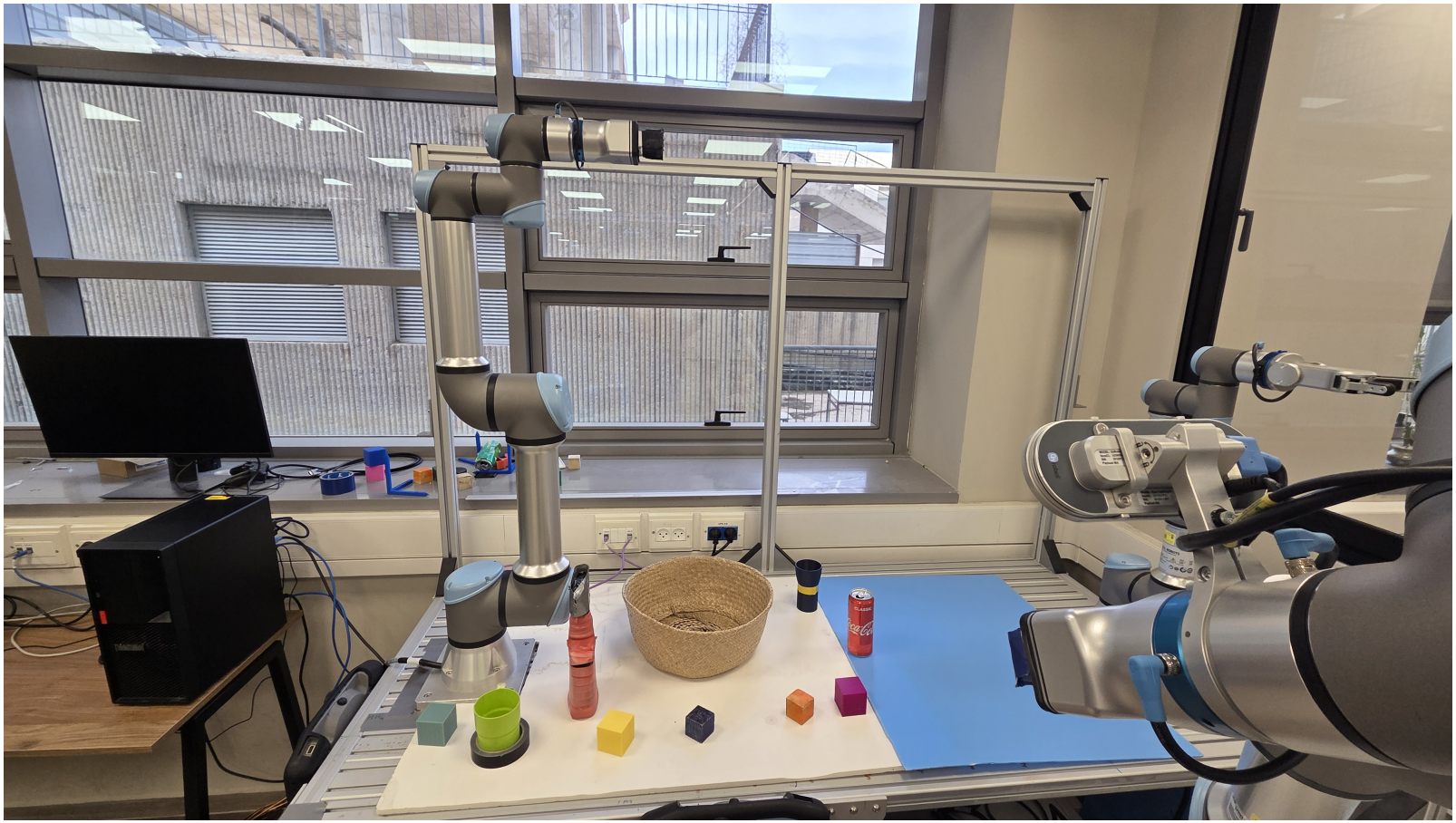
The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

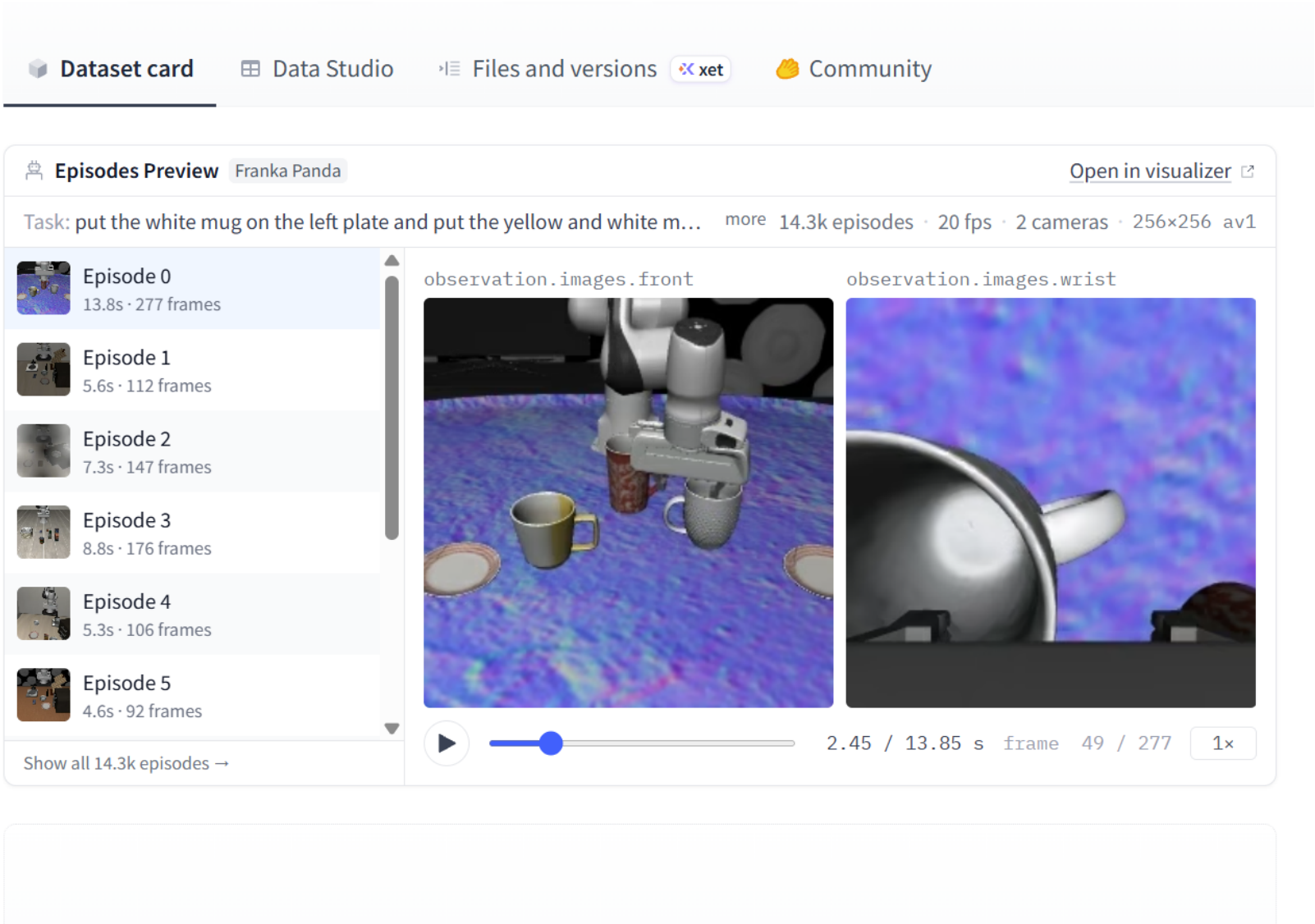
’go from A to B’?


ANTENNA FOR RADIO LINK
TELEVISION CAMERA
RANGE FINDER
ON-BOARD LOGIC
CAMERA CONTROL UNIT
BUMP DETECTOR
CASTER WHEEL
DRIVE MOTOR
DRIVE WHEEL


A
B
C
D
E
F
G
H
I
J







https://huggingface.co/datasets/lerobot/libero_plus

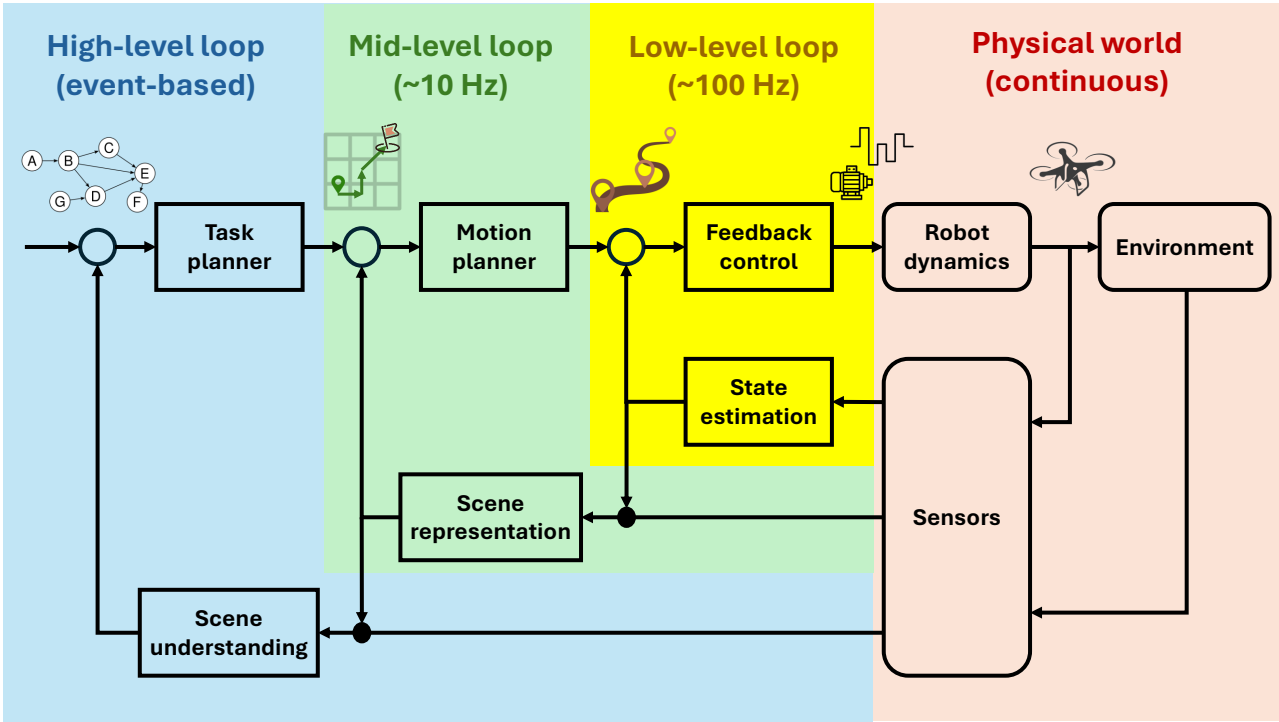
'Pick up the tennis ball'

'Pick up the tennis ball'
'Pick up an apple'

'Pick up the tennis ball'
'Pick up an apple'
'I am hungry'

Layered Decision-Making

Layered Decision-Making

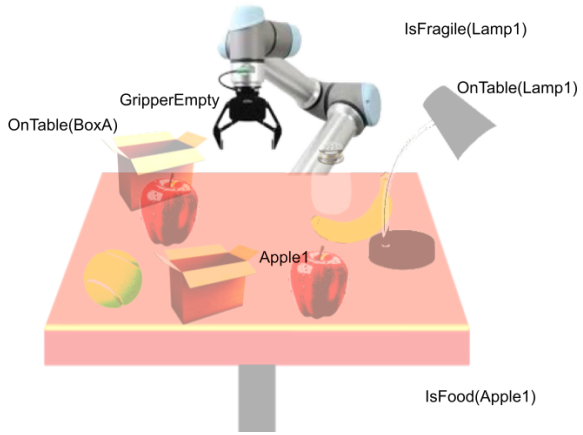


Layered Decision-Making
Task Planning
Motion Planning
Control

Task Interpretation - Task Planning

Layered Decision-Making
Task Planning
Motion Planning
Control

Translate the scene into a symbolic representation



The representation (e.g., using PDDL or STRIPS domain) is designed to be used by the high-level symbolic planner that finds a sequence of actions or policy to achieve the goal.

Task Planning

A planner translates the goal condition (“pick up an apple”) into a concrete sequence of actions.
Possible representation:

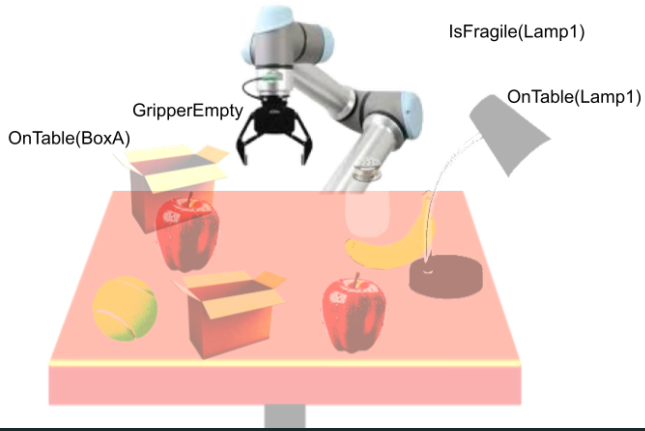
Goal: (holding(apple) = true)
State Features: apple_on(table), gripper_empty ...
Actions: move_to(apple), grasp(apple)...

Is this a sufficient representation for our setting ? What if we have more than one way to achieve the goal (here, more than one apple)?

Layered Decision-Making
Task Planning
Motion Planning
Control

Task Planning

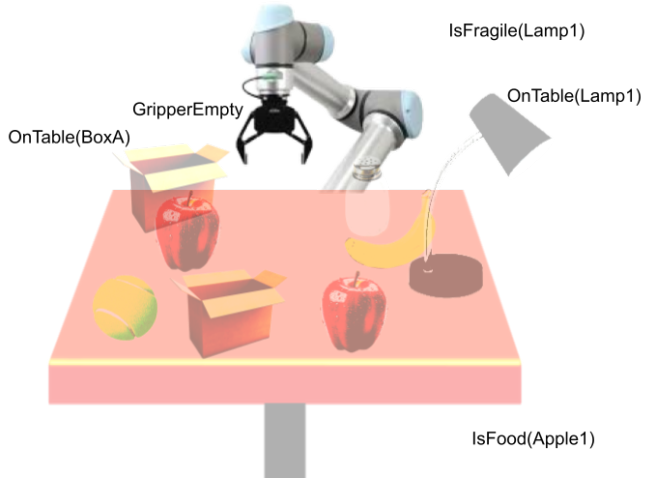
Let $\mathcal{R}$ be a robot operating in a discrete environment described by a set of symbolic **states** $\mathcal{S}$ and **actions** $\mathcal{A}$. Given an initial state $s_0 \in \mathcal{S}$ and a **goal condition** $G \subseteq \mathcal{S}$, determine whether there exists a sequence of actions $\pi = \langle a_1, a_2, \ldots, a_n \rangle$, such that executing π from s_0 leads to a state $s_n \in G$.



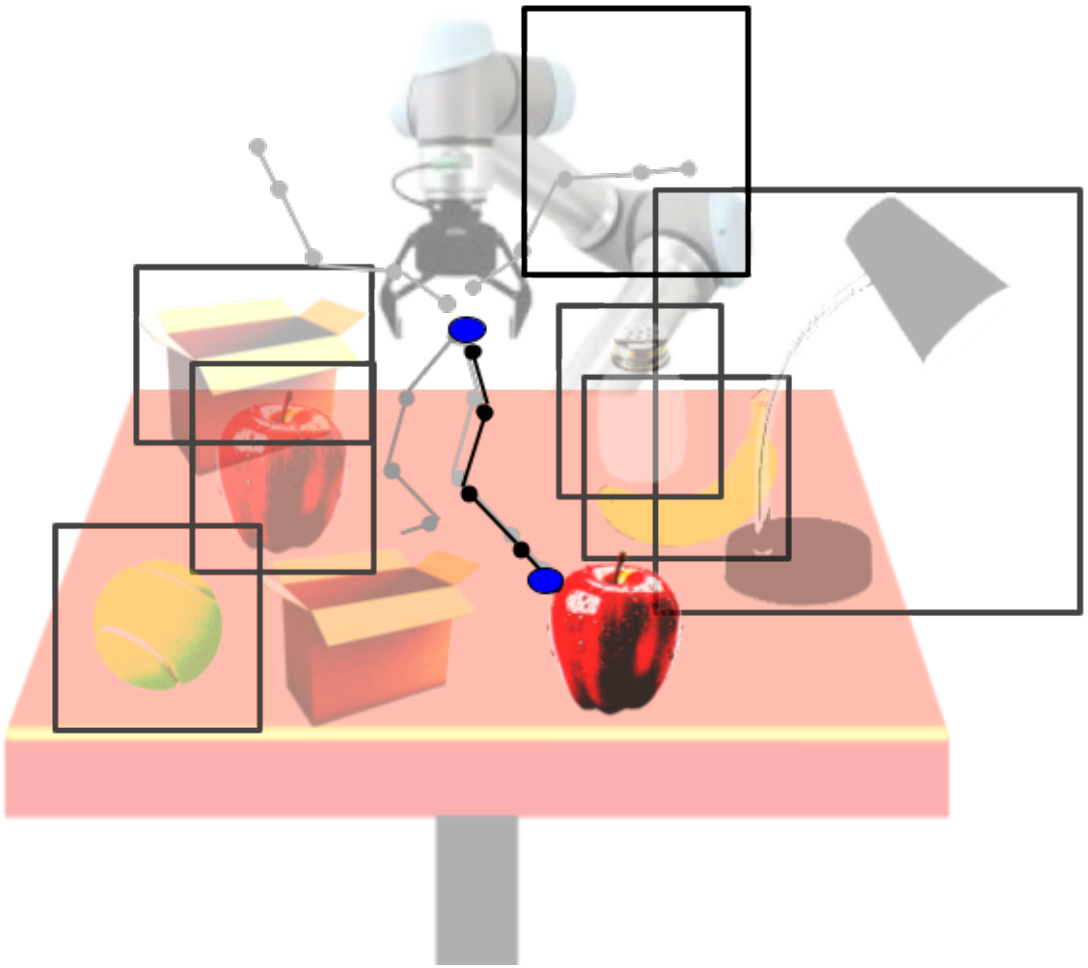
Layered Decision-Making
Task Planning
Motion Planning
Control

Task Planning

- **Input:** Goal condition: 'pick up an apple'
- **Output:** A sequence of (high-level) actions that achieve the goal
- **Option:** use propositional STRIPS to define the planning task as $P = \langle \mathcal{F}, I, \mathcal{A}, G, \mathcal{C} \rangle$ and use A* to solve the problem



Motion Planning



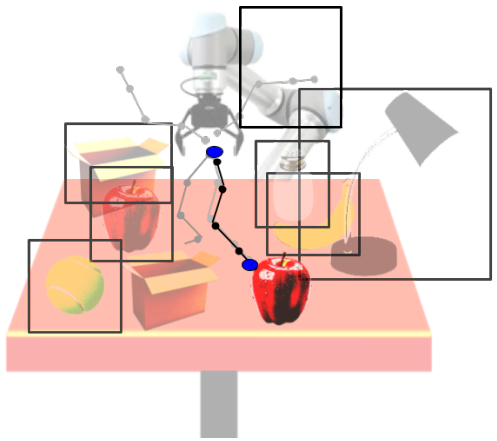
Layered Decision-Making
Task Planning
Motion Planning
Control

Motion Planning

A motion planner is used to compute a **collision-free trajectory** for the robot's arm to reach the apple.

The planner ensures:

- desired position is reached (the gripper's approach vector aligns with the apple)
- no collision occurs with the table or nearby objects.
- kinematic feasibility (joint limits, reachability, etc.) is satisfied.

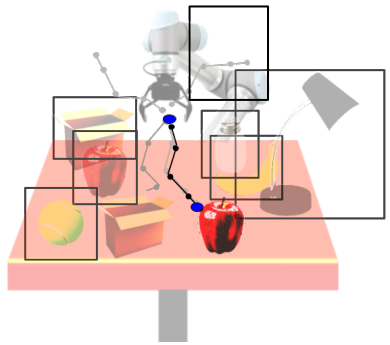


Layered Decision-Making
Task Planning
Motion Planning
Control

Motion Planning

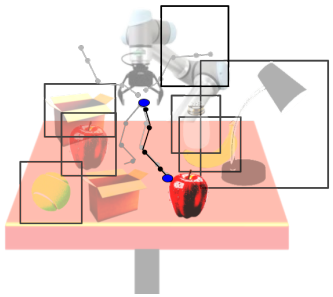
Layered Decision-Making
Task Planning
Motion Planning
Control

- **Input:** Initial and goal configurations and map of (obstacles in the) environment.
- **Output:** A collision-free path from the initial to the goal configuration.



Motion Planning

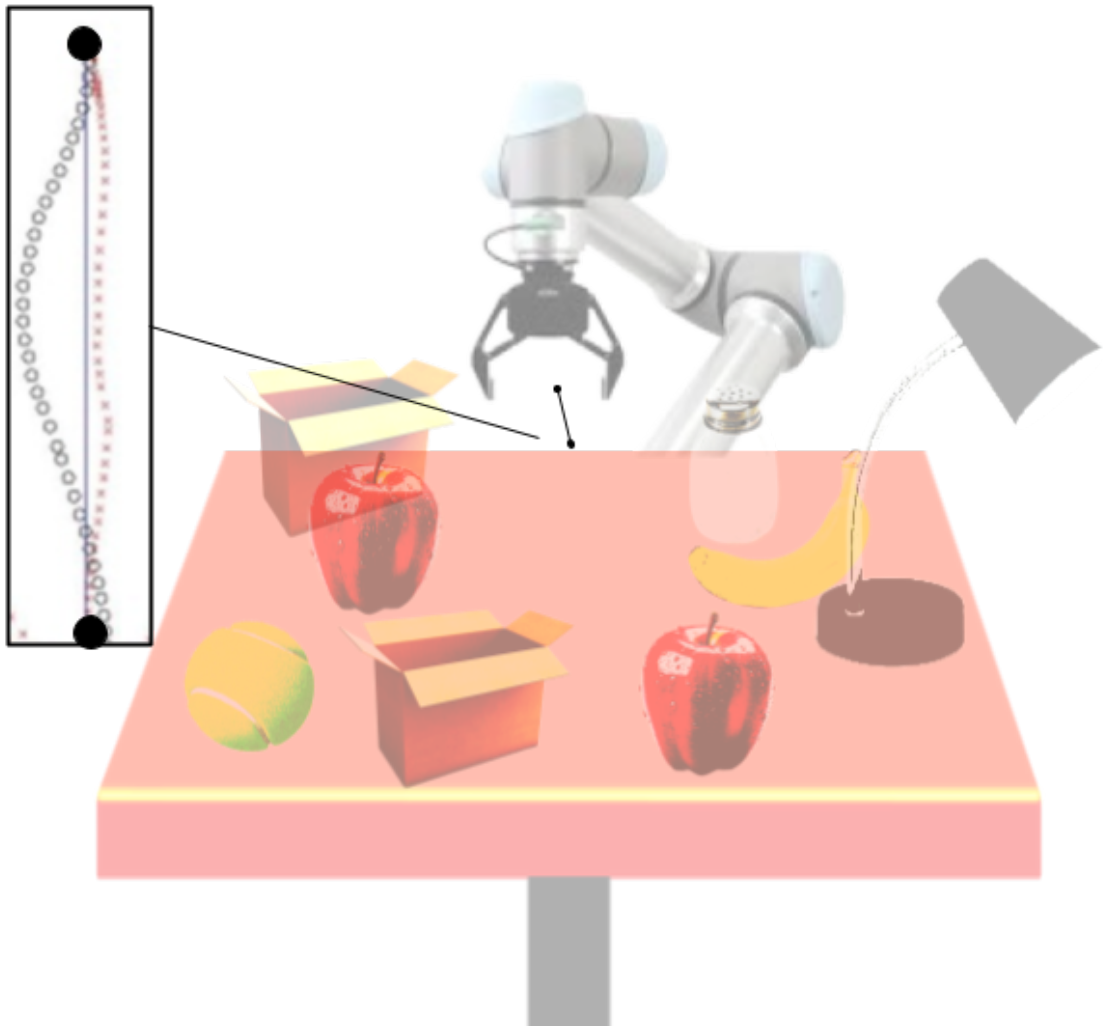
Let $\mathcal{R}$ be a robot system with d **degrees of freedom**, moving in a known environment with obstacles. Given start and target **configurations** q_0 and q_g for $\mathcal{R}$, decide whether there is a collision-free, continuous path $\tau : [0, 1] \to \mathcal{C}_{free}$ such that $\tau(0) = q_0$ and $\tau(1) = q_g$ and if so, plan such a motion.



Solution Approach: map the environment into free and occupied space and use A* over the free space.

Layered Decision-Making
Task Planning
Motion Planning
Control

Motion Control

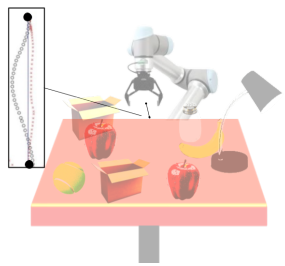


Layered Decision-Making
Task Planning
Motion Planning
Control

Motion Control

”Move the arm, close the gripper, and lift the apple”

- **Trajectory Execution** - low-level controller (e.g., PID, MPC) uses feedback from joint encoders and force sensors to follow the planned trajectory.
- **Force control loop** - tactile sensors measure contact pressure so that the controller can regulate the grasp force.
- **Vision-based feedback** - refines the alignment if necessary; the apple disappears from the table view and appears in gripper view.
- **Post-Grasp Motion** - The robot lifts the apple slightly and moves to a safe “retract” position.

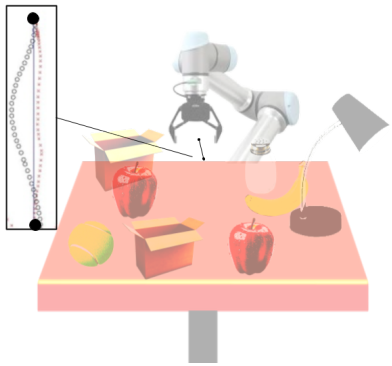


Layered Decision-Making
Task Planning
Motion Planning
Control

Layered Decision-Making
Task Planning
Motion Planning
Control

Motion Control

Given a desired trajectory $\tau(t)$ for a robotic system $\mathcal{R}$ with d **degrees of freedom**, the goal of motion control is to compute and apply control inputs $\mathbf{u}(t)$ (e.g., joint torques, velocities, or forces) such that the system's actual motion $q(t)$ follows $\tau(t)$ as closely as possible.



Motion Control

- **Input:** Desired trajectory
- **Output:** A control procedure that minimizes the deviation between the executed and desired trajectory.

System dynamics are captured by $\dot{x}(t) = f(x(t), u(t), t) + w(t)$ where

- $x(t) \in \mathbb{R}^n$ - state vector
- $u(t) \in \mathbb{R}^m$ - control input
- $f : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R} \to \mathbb{R}^n$ - system dynamics
- $w(t)$ - noise or disturbance

The challenge is to translate actions into low-level commands (and monitor their execution).

From Instructions to Torques: ’pick up an apple’

State estimation and decision making at each step.

Layered Decision-Making
Task Planning
Motion Planning
Control

From Instructions to Torques: ’pick up an apple’

Layered Decision-Making
Task Planning
Motion Planning
Control

From Instructions to Torques: ’pick up an apple’

- `pick_up_apple`
  - **State Estimation:** locate an apple in the scene
  - **Decision Making:** decide which apple to try and pick up

From Instructions to Torques: 'pick up an apple'

- `pick_up_apple`
  - **State Estimation:** locate an apple in the scene
  - **Decision Making:** decide which apple to try and pick up
- `move_to_grasp_config`
  - **State Estimation:**
    - **offline:** mark all the obstacles in the environment (including the robot) in the configuration space.
    - **online:** detect robot position (in dynamic environments, also the position of obstacles)
  - **Decision Making:** decide how to reach the configuration

From Instructions to Torques: 'pick up an apple'

- `pick_up_apple`
  - **State Estimation:** locate an apple in the scene
  - **Decision Making:** decide which apple to try and pick up
- `move_to_grasp_config`
  - **State Estimation:**
    - **offline:** mark all the obstacles in the environment (including the robot) in the configuration space.
    - **online:** detect robot position (in dynamic environments, also the position of obstacles)
  - **Decision Making:** decide how to reach the configuration
- `follow_trajectory`
  - **State Estimation:** monitor robot location w.r.t. reference trajectory
  - **Decision Making:** corrective actions

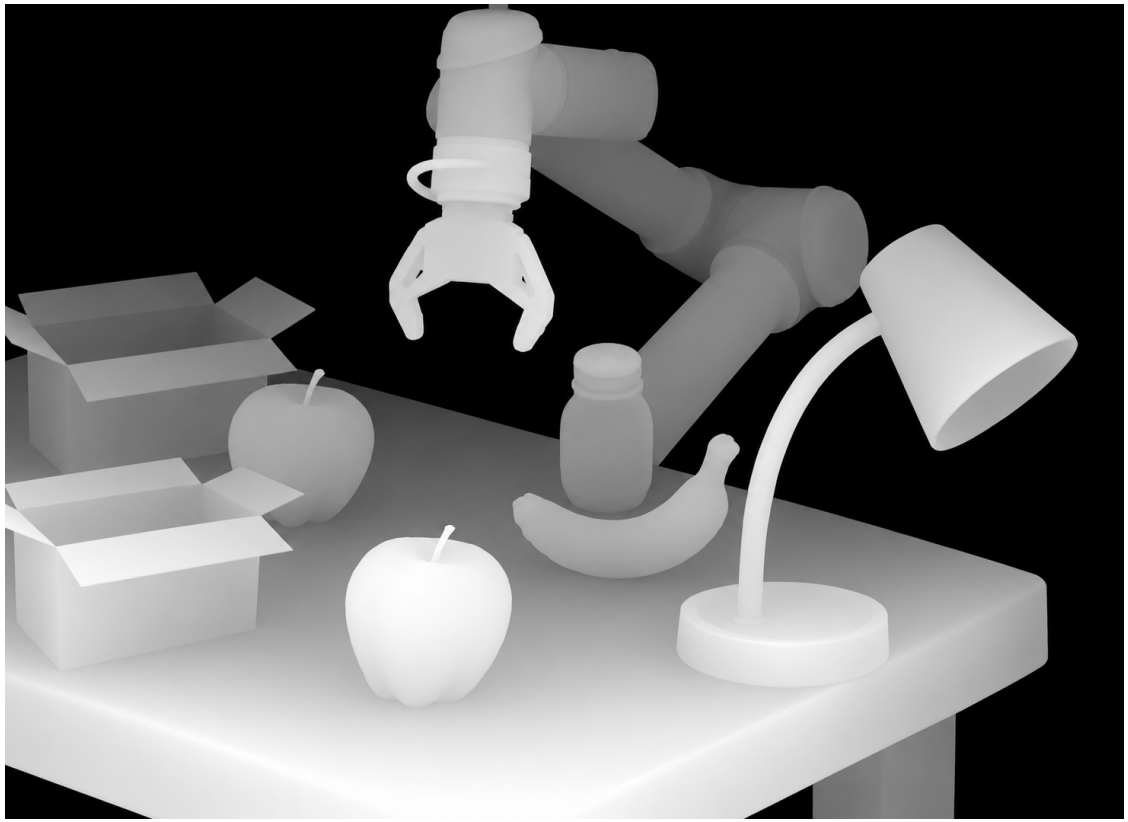
Perception and Scene Understanding

Layered Decision-Making
Task Planning
Motion Planning
Control

Perception and Scene Understanding

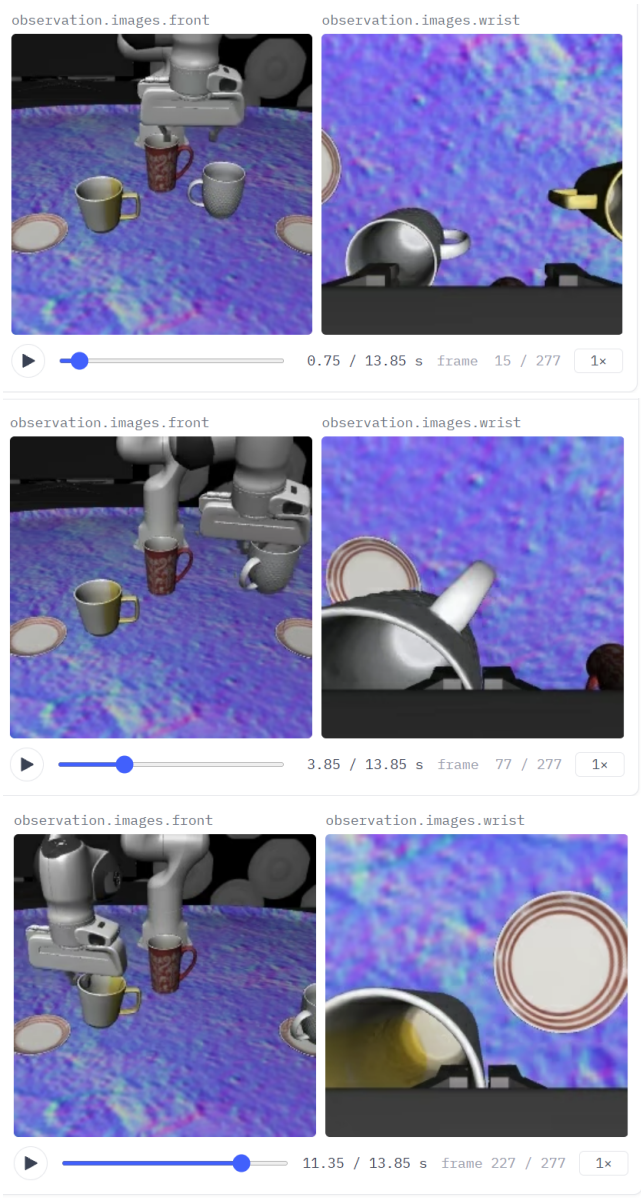
Layered Decision-Making
Task Planning
Motion Planning
Control

Perception and Scene Understanding



Layered Decision-Making
Task Planning
Motion Planning
Control

Perception and Scene Understanding



Layered Decision-Making

Task Planning

Motion Planning

Control

Perception and Scene Understanding

Sense and interpret the environment to locate an apple and estimate its properties.

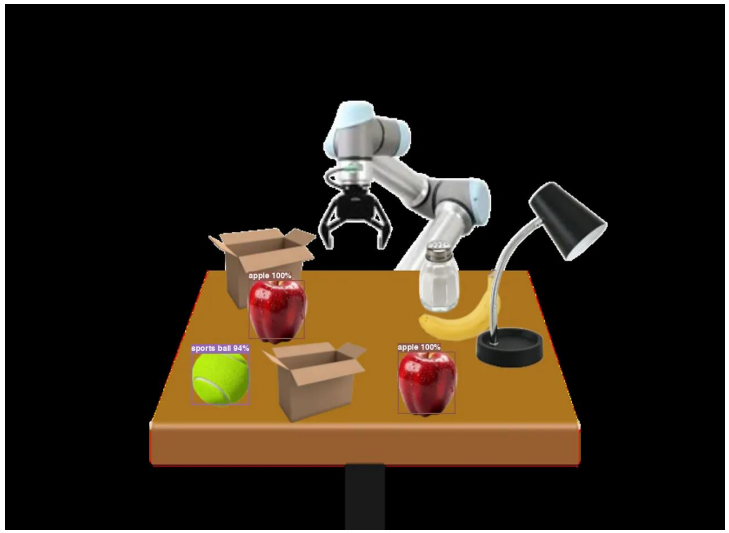
Sensor Data Acquisition: RGB-D camera (color + depth), LiDAR, stereo camera, tactile and proximity sensors...

`https://huggingface.co/spaces/nielsr/dpt-depth-estimation`

Perception and Scene Understanding

Layered Decision-Making
Task Planning
Motion Planning
Control

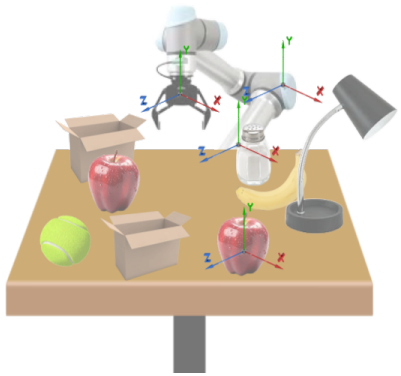
Object Detection and Segmentation: detect and classify objects in the image



YOLO v3: `https://v-iashin.github.io/detector`

Perception and Scene Understanding

- **3D Localization:** Using the depth map, the robot computes the apple's 3D position in its camera coordinate frame.
- This point is transformed into world coordinates using the robot's calibration data.
- Uncertainty of estimation influences planning (e.g., grasp robustness).



Uncertainty Management and Error Handling

- If the apple is occluded, the robot performs active exploration (moves head/camera).
- If the robot's gripper misses the apple, it may trigger a failure recovery behavior (e.g., retry with a new pose estimate).
- If grasp fails (slip detected or object dropped), the system may replan or retry.

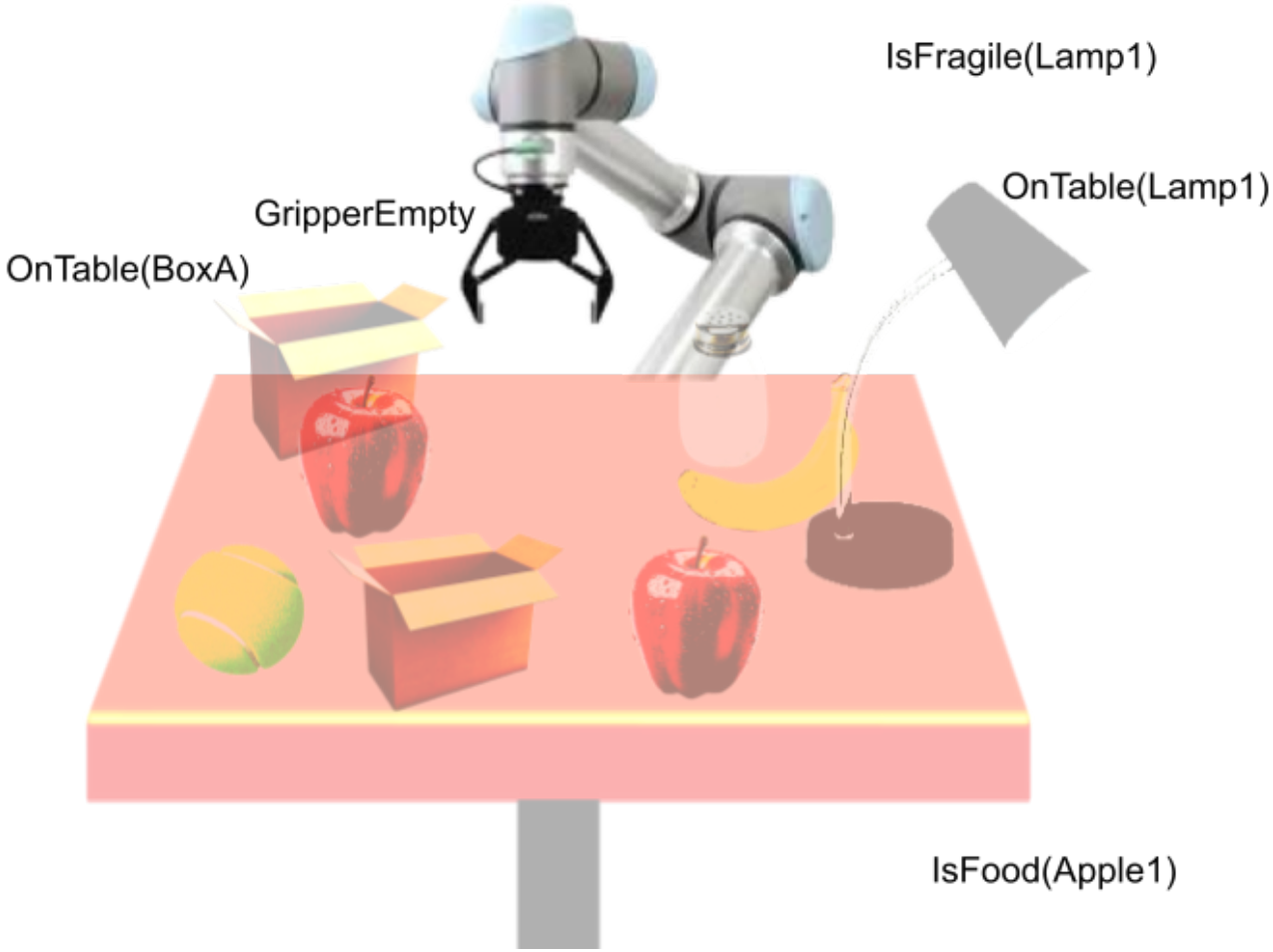
Learning and Adaptation

The objective is to improve performance over time.

- **Reinforcement Learning:** The robot can learn grasp strategies through trial and error based on success rates.
- **Imitation Learning:** The robot can learn from demonstrations of humans picking apples.
- **Active Perception:** If uncertainty in pose estimation is high, the robot may reposition its camera to obtain a better view before acting.

Task Planning

Task Planning



Layered Decision-Making
Task Planning
Motion Planning
Control

Task Planning

Layered Decision-Making
Task Planning
Motion Planning
Control

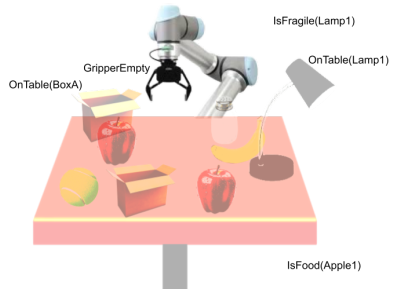
Goal: Find a sequence of actions that transforms an initial state into a state satisfying a desired goal.

A planning problem specifies:

- **States**: relevant properties of the world.
- **Actions**: their preconditions and effects.
- **Goal**: conditions that must eventually hold.

Task Planning

Let $\mathcal{R}$ be a robot operating in a discrete environment described by a set of symbolic **states** $\mathcal{S}$ and **actions** $\mathcal{A}$. Given an initial state $s_0 \in \mathcal{S}$ and a **goal condition** $G \subseteq \mathcal{S}$, determine whether there exists a policy $\pi : \mathcal{A} \times \mathcal{S} \mapsto [0, 1]$ such that executing π from s_0 leads to a state $s_n \in G$.

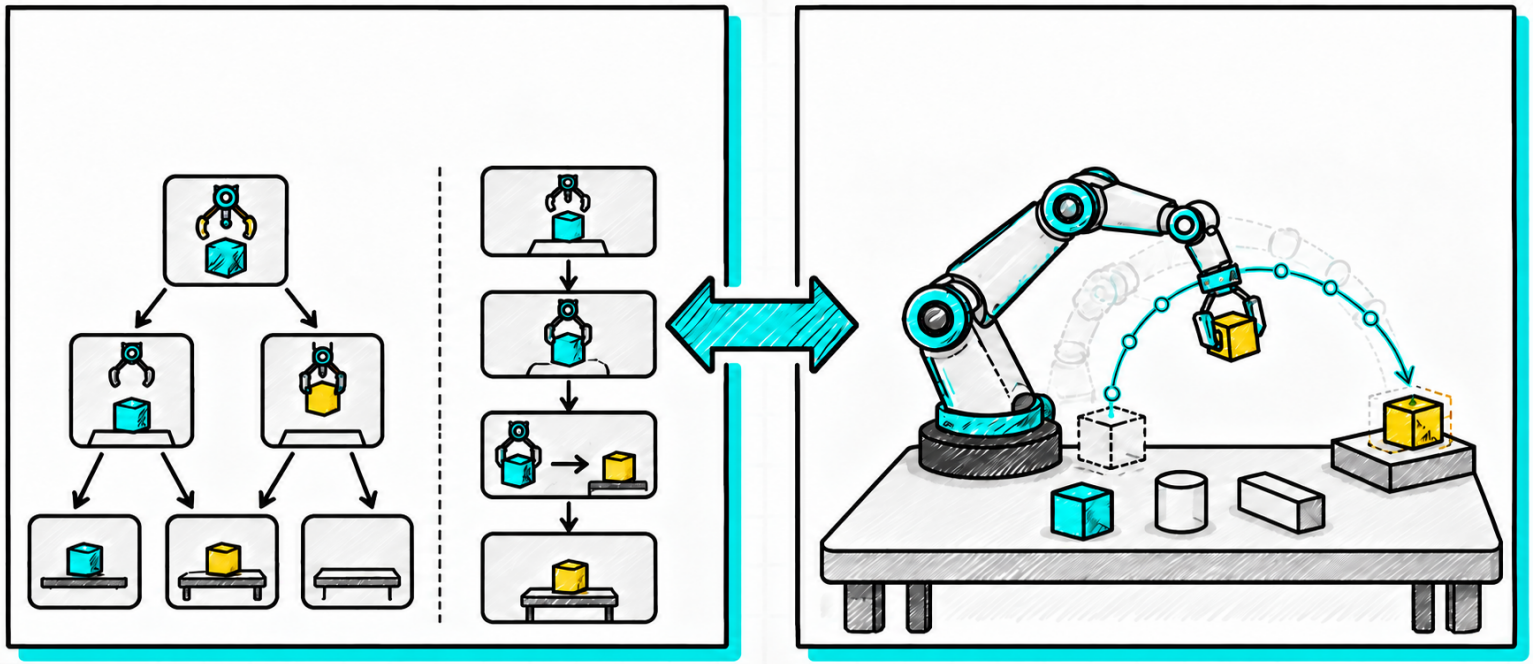


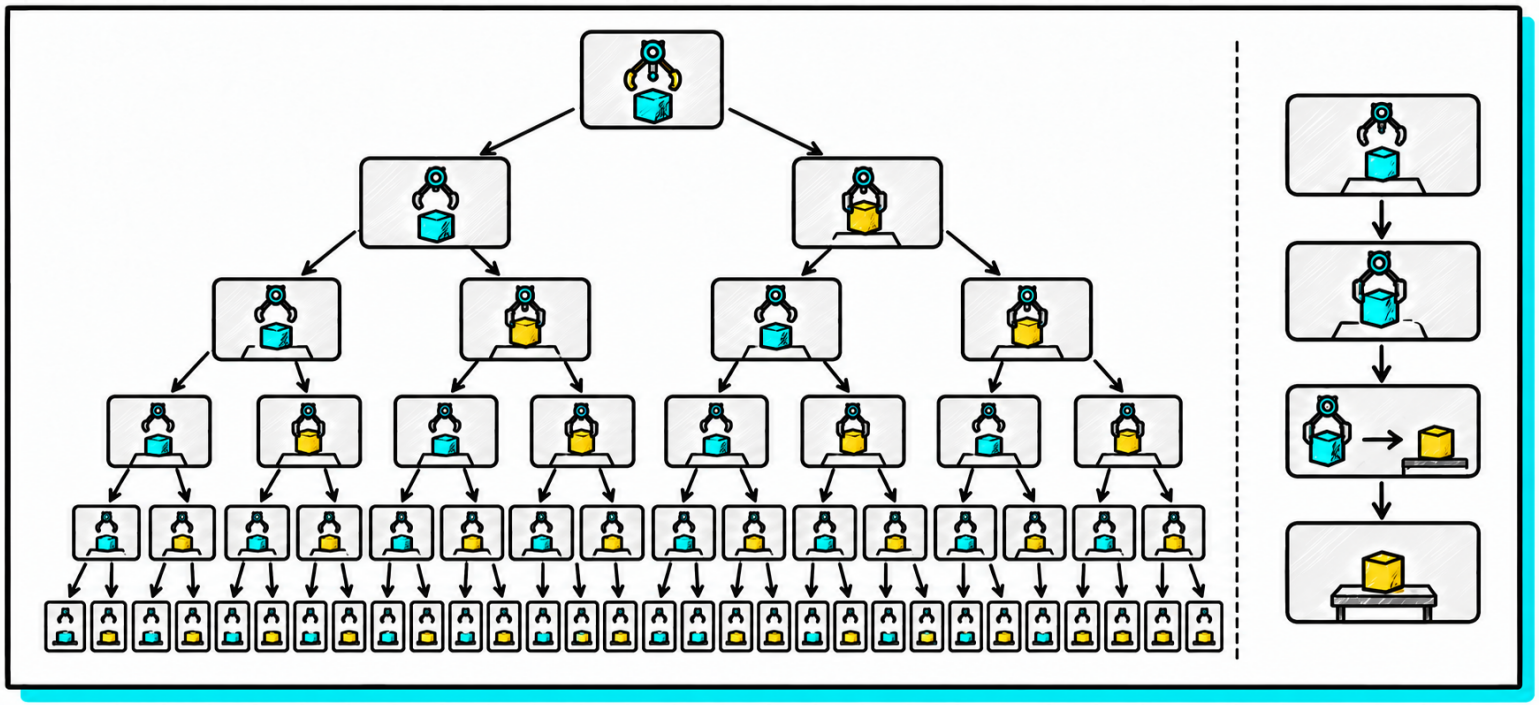
What are the underlying assumptions in this definition?

Layered Decision-Making
Task Planning
Motion Planning
Control

Why It Is Hard

- **Combinatorial search**: with b choices per step, there can be b^d action sequences of length d.
- **Interacting decisions**: an action may enable one goal while making another harder to achieve.
- **Long-term dependencies**: useful actions may require many preparatory steps.





Planning as General Problem Solving

- Block stacking
- Logistics
- Mars rovers
- Cyber security
- Natural language
- ...

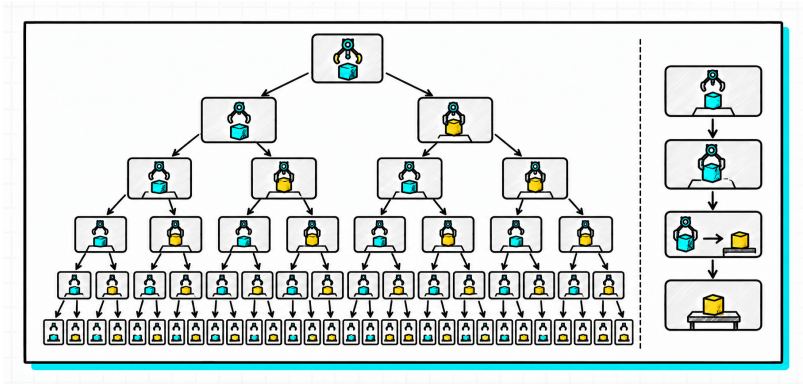
Types of Planning Problems

- **Classical**: discretized world, finite domains, deterministic action effects, single agent of change.
- **Numeric**: domain allows continuous values (e.g.: x, y positions)
- **Temporal**: actions take time, parallel actions, goals can have deadlines (e.g., walking will take 5-10 minutes, I need to be there in 1 hour)
- **Multi-agent**: planning for multiple coordinating agents, or against an adversary (e.g., multiple UAVs, or planning against cyber-attack)
- **Non-deterministic:** actions can have uncertain effects, uncertain durations. (e.g., driving will have a 99% of success of reaching your destination, but a 1% chance of an accident)
- ...

Basic Approach: State Space Search

Represent the planning problem as a **search graph**:

- **Nodes**: states of the world.
- **Edges**: applicable actions, with associated costs.
- **Solution**: a path from the initial state to a goal state.



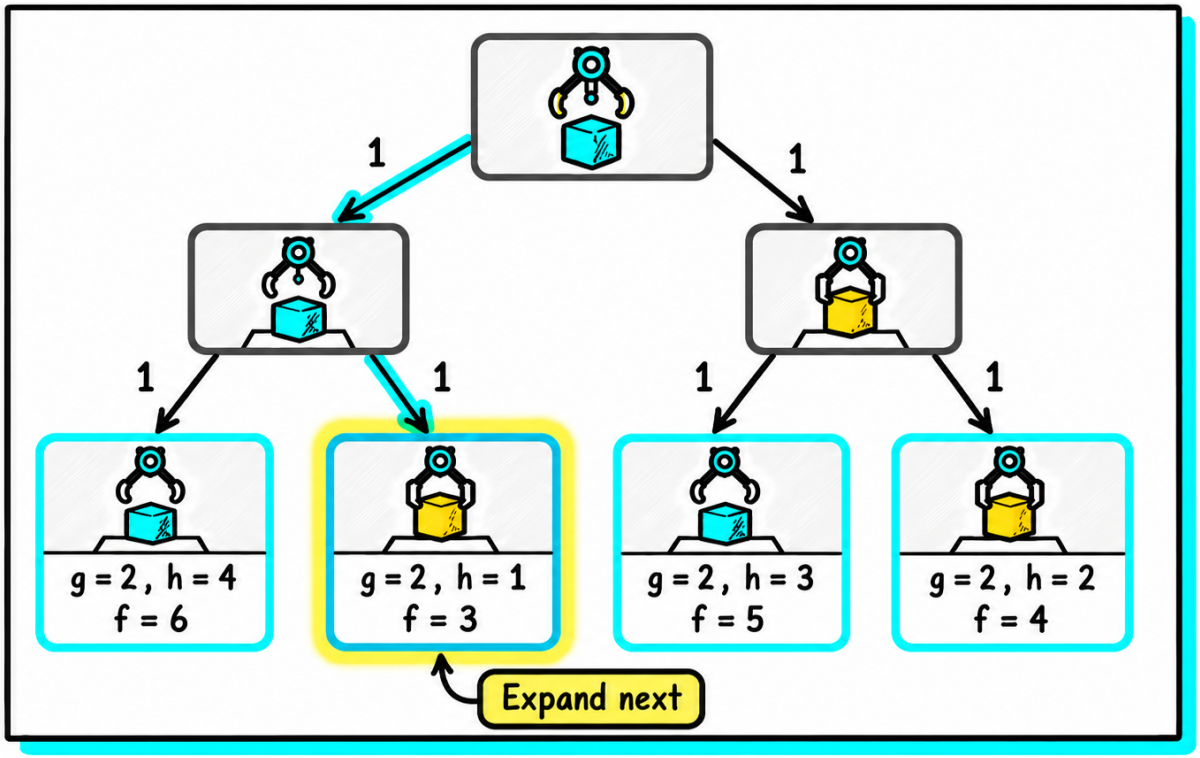
Basic Approach: A*

A* expands the frontier node with the smallest estimated total cost:

$$f(n) = \underbrace{g(n)}_{\text{cost so far}} + \underbrace{h(n)}_{\text{estimated cost to a goal}} .$$

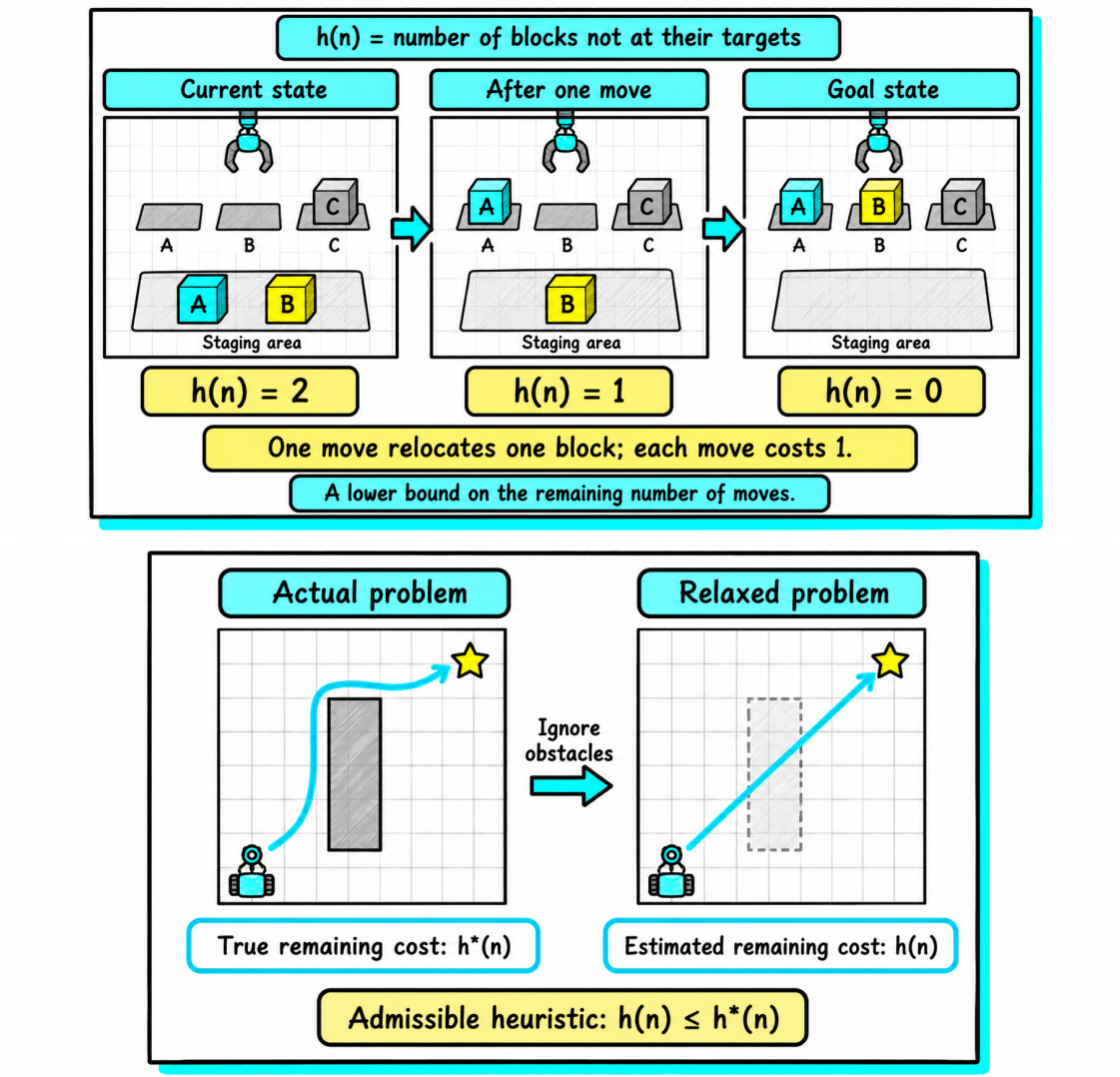
- **Search**: expand a node, generate successors, and update their best known costs.
- **Stop**: when a goal node is removed from the frontier.
- **Optimality**: with an admissible heuristic $(h(n) \leq h^*(n))$, A* finds a least-cost plan.

A*



Layered Decision-Making
Task Planning
Motion Planning
Control

Heuristics



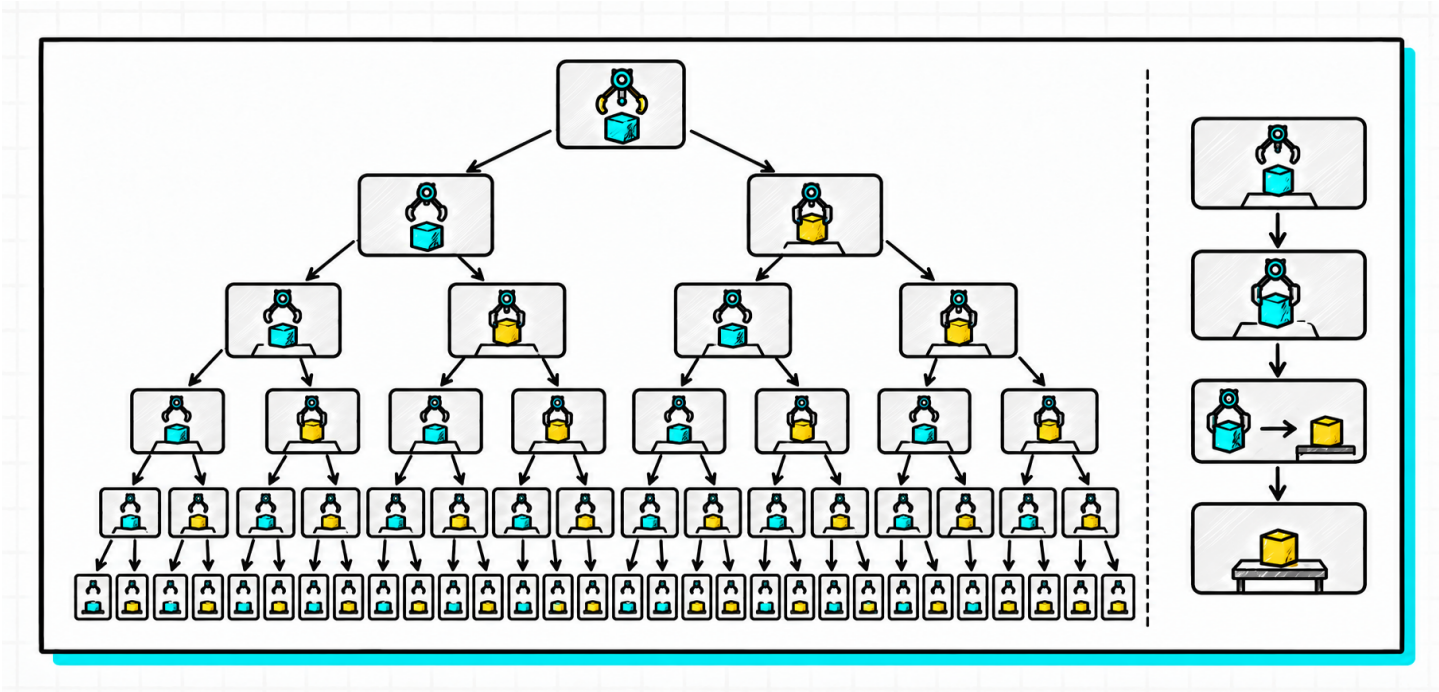
Layered Decision-Making

Task Planning

Motion Planning

Control

Challenges



Main challenge: Even with a heuristic, the number of explored states and memory requirements can grow exponentially.

Layered Decision-Making
Task Planning
Motion Planning
Control

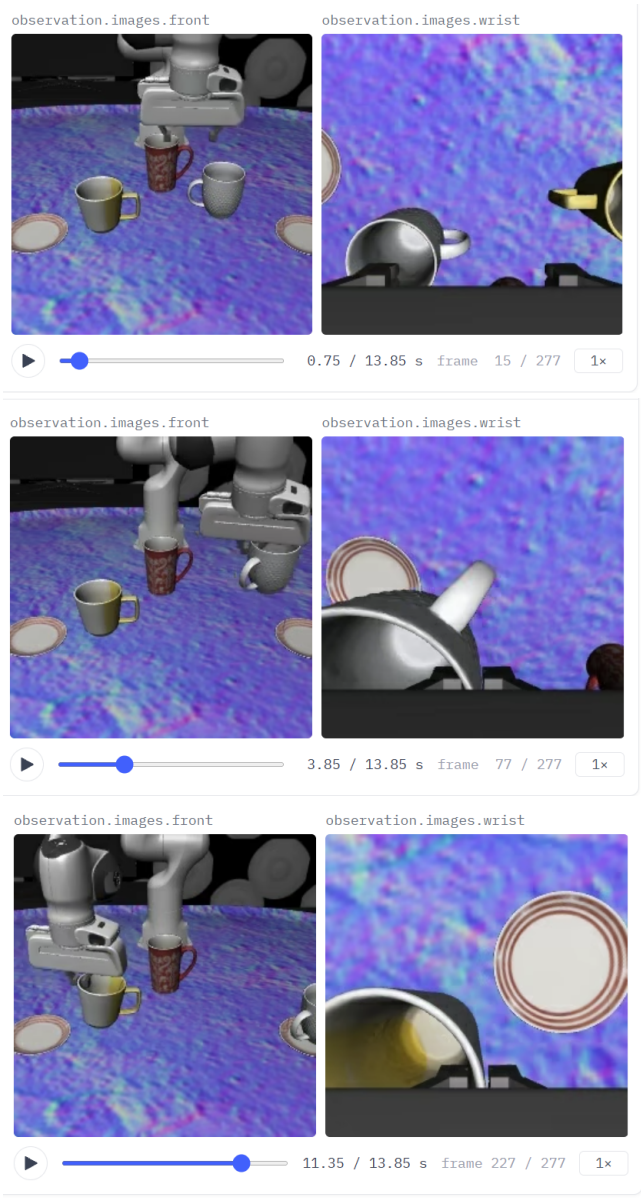
Optimization Considerations

- **Completeness**: does the algorithm find a solution when there is one?
- **Soundness:** if a solution is found, is it a valid solution?
- **Optimality:** is the solution a maximal-utility / minimal-cost solution?
- **Generality:** what are the assumptions about the domain that guarantee completeness, optimality, etc.
- **Resource consumption**
  - **Time complexity:** how long does it take to find a solution (e.g., in terms of node expansions)?
  - **Space complexity:** how much memory is needed to perform the search (e.g., in terms of nodes and edges preserved in memory)?
- **Anytime guarantees:** with more resources, is the solution guaranteed not to degrade?

Types of Planning Problems: Objective

- **Optimization focus:** What are we minimizing or maximizing?
  - number of actions
  - time spent
  - fuel used
  - utility
- **Bounded Suboptimality** e.g., find a solution which is at most twice as expensive as an optimal one
- **Satisficing** e.g., find a "good enough" solution.
- **Preferences** e.g., express soft goals, preferred actions, or action ordering. "I prefer to avoid a main street on the way to my friend's house."

Why Task Planning Matters in Robotics



Layered Decision-Making

Task Planning

Motion Planning

Control

Why Task Planning Matters in Robotics

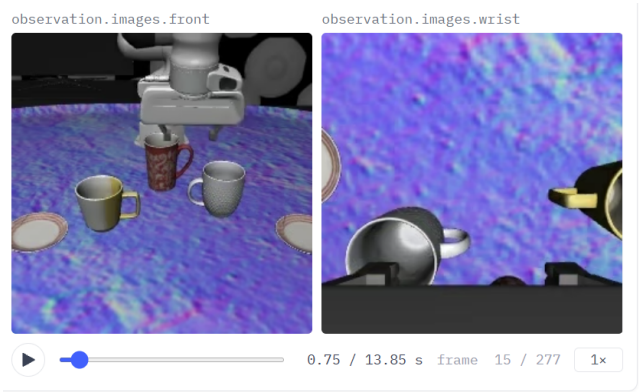
Robots must translate **high-level goals** into **coordinated sequences of physical actions**.

Example: Place a cup in a cupboard.

Open the door $\rightarrow$ Pick up the cup $\rightarrow$ Place it inside $\rightarrow$ Close the door

Example: Put away the dishes.

Put away cup1 $\rightarrow$ Put away cup2 $\rightarrow$ Put away plate $\rightarrow$ $\cdots$



Layered Decision-Making
Task Planning
Motion Planning
Control

Why Task Planning Matters in Robotics

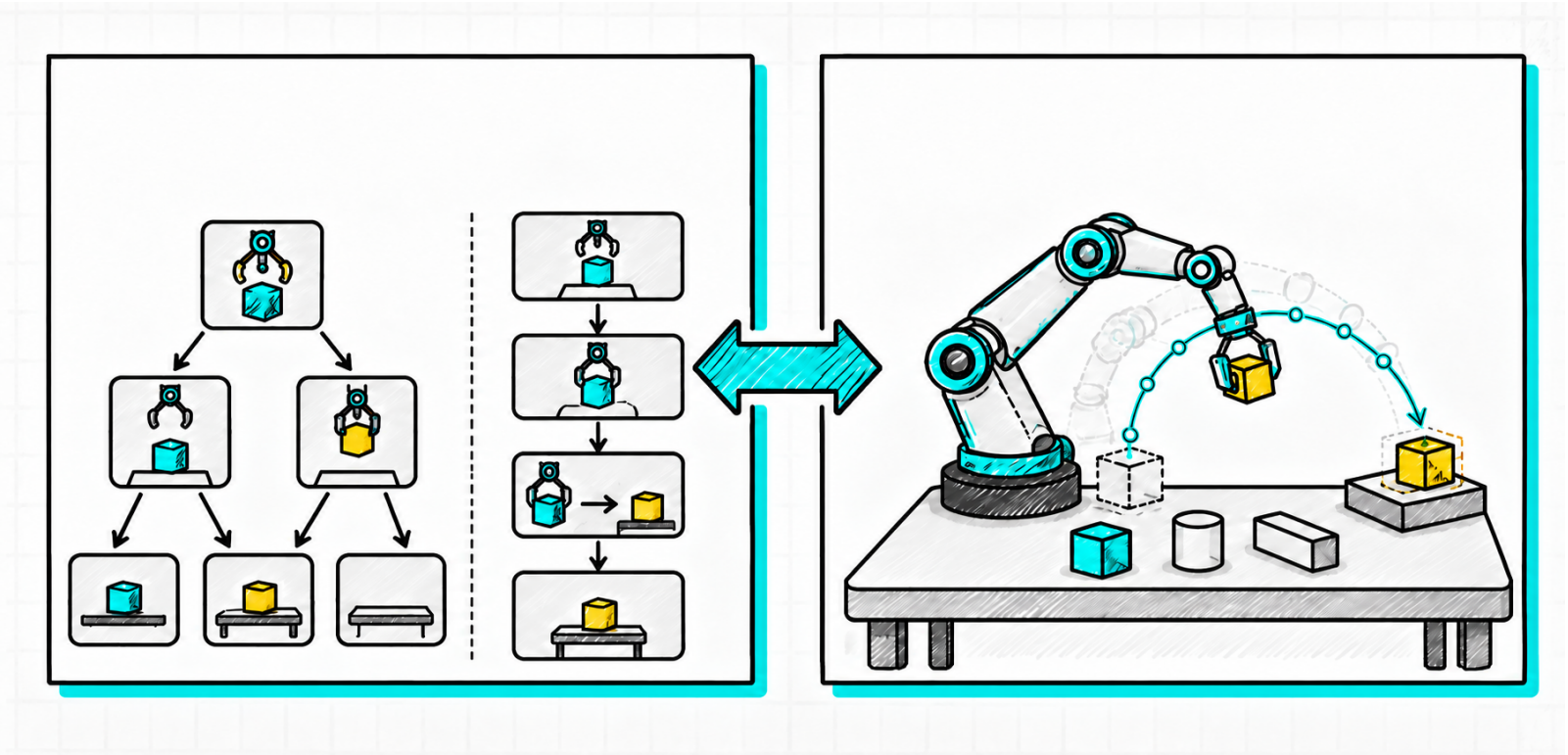
Task planning helps a robot:

- **Respect dependencies**: open the cupboard before placing the cup inside.
- **Choose among alternatives**: select an object, a destination, or an action order.
- **Coordinate capabilities**: combine navigation, grasping, and manipulation.
- **Recover from failure**: replan when observations or execution outcomes invalidate the plan.

The robotics challenge: A logically valid plan must also be physically feasible.

Why Task Planning Matters in Robotics

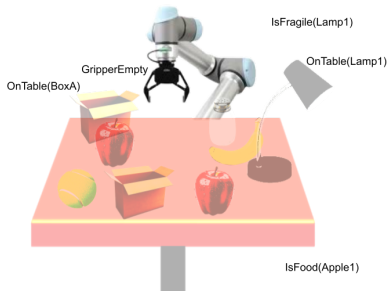
The robotics challenge: A logically valid plan must also be physically feasible.



Layered Decision-Making
Task Planning
Motion Planning
Control

Task Planning

Let $\mathcal{R}$ be a robot operating in a discrete environment described by a set of symbolic **states** $\mathcal{S}$ and **actions** $\mathcal{A}$. Given an initial state $s_0 \in \mathcal{S}$ and a **goal condition** $G \subseteq \mathcal{S}$, determine whether there exists a policy $\pi : \mathcal{A} \times \mathcal{S} \mapsto [0, 1]$ such that executing π from s_0 leads to a state $s_n \in G$.



Isn't Planning Enough?

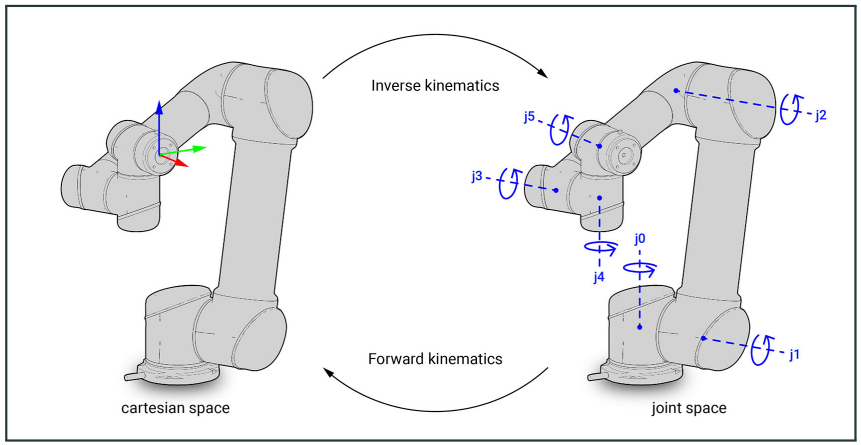
What are the underlying assumptions in this definition?

Layered Decision-Making
Task Planning
Motion Planning
Control

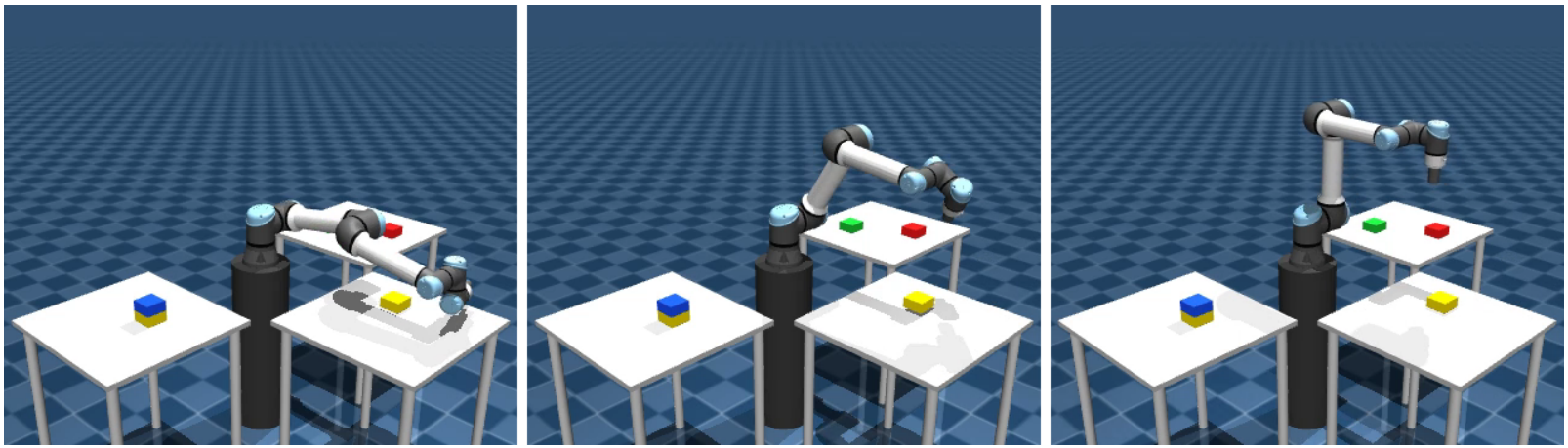
Motion Planning

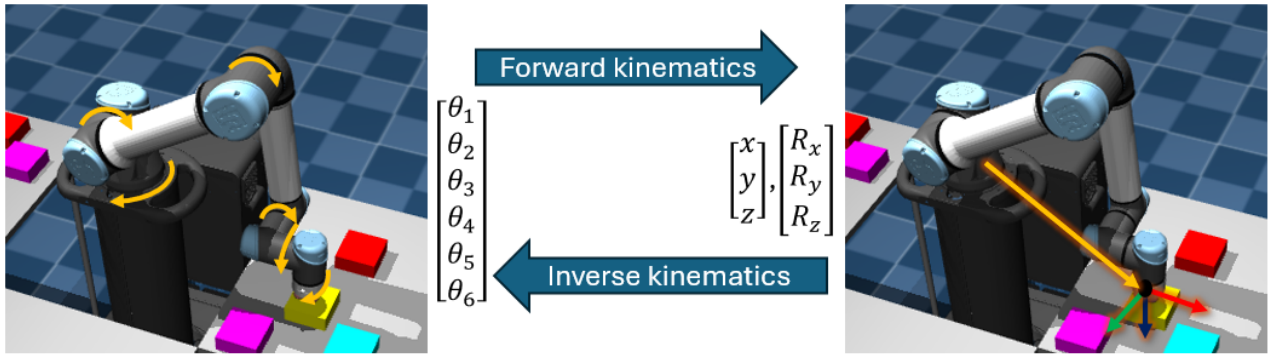
Forward and Inverse Kinematics

- **Forward Kinematics** - the use of the kinematic equations of a robot to compute the position of the end-effector from specified values for the joint parameters: **what will be the robot's configuration if it executes a given command sequence ?**
- **Inverse Kinematics** - the use of kinematic equations to determine the motion of a robot to reach a desired position: **what is the command sequence that will get my robot to a desired configuration ?**

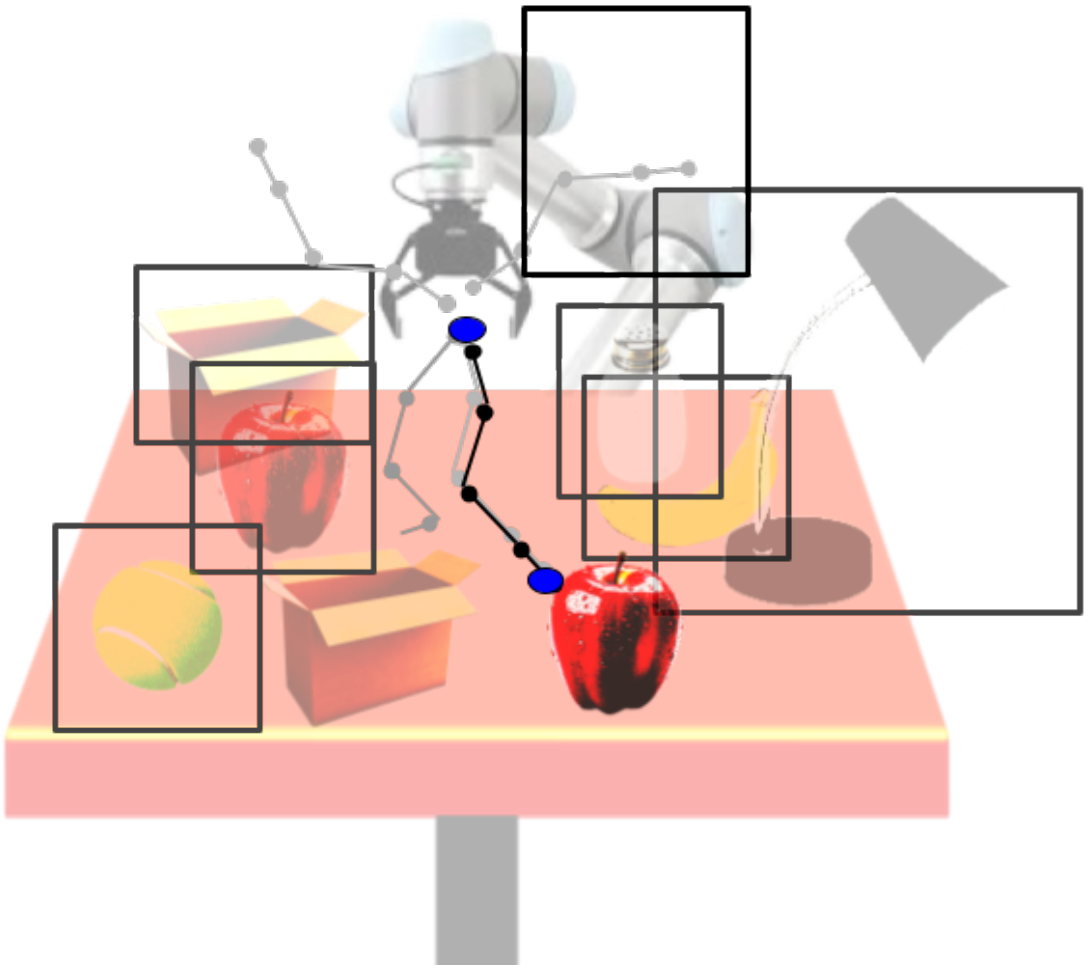


Isn’t Inverse Kinematics Enough?





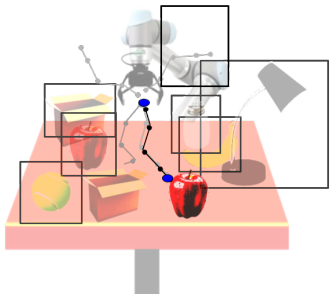
Motion Planning



Layered Decision-Making
Task Planning
Motion Planning
Control

Motion Planning

Let $\mathcal{R}$ be a robot system with d **degrees of freedom**, moving in a known environment with obstacles. Given start and target **configurations** q_0 and q_g for $\mathcal{R}$, decide whether there is a collision-free, continuous path $\tau : [0, 1] \to \mathcal{C}_{free}$ such that $\tau(0) = q_0$ and $\tau(1) = q_g$ and if so, plan such a motion.



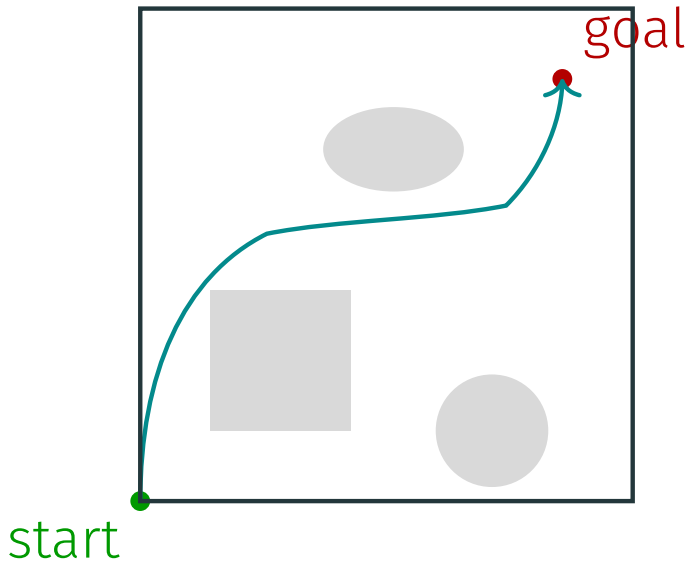
Solution Approach: map the environment into free and occupied space and use A* over the free space.

Layered Decision-Making
Task Planning
Motion Planning
Control

Motion Planning for a Point Robot

Goal: Move a point robot from start to goal without collisions.

- Robot is modeled as a point in $\mathbb{R}^2$ or $\mathbb{R}^3$.
- Environment contains obstacles $O_1, O_2, \ldots, O_n$.
- **Free space:** $\mathcal{C}_{free} = \mathcal{C} \setminus \bigcup_i O_i$.
- **Goal:** Find continuous path $\tau : [0, 1] \to \mathcal{C}_{free}$ with $\tau(0) = q_{\text{start}}$, $\tau(1) = q_{\text{goal}}$.

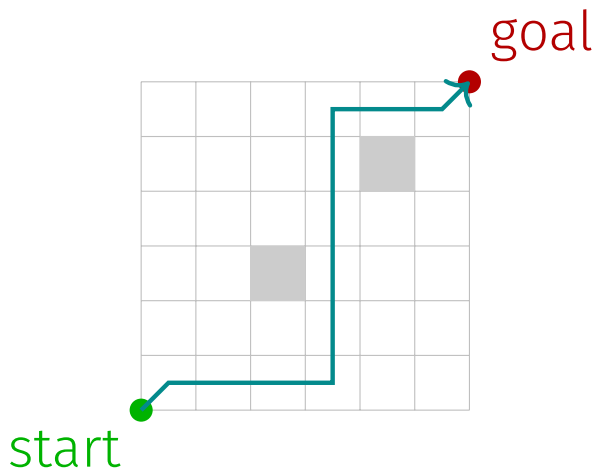


Search-Based Motion Planning

Idea: Discretize $\mathcal{C}_{free}$ into a grid and search for a path.

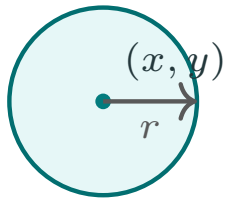
Typical methods:

- **Breadth-first search (BFS)** — shortest path on uniform grid.
- **A* search** — uses heuristic $h(q)$ to guide exploration.



Advantages: simple, optimal on grid.
Limitations: exponential growth with dimension.

How do we plan for a disc robot?



Disk Robot

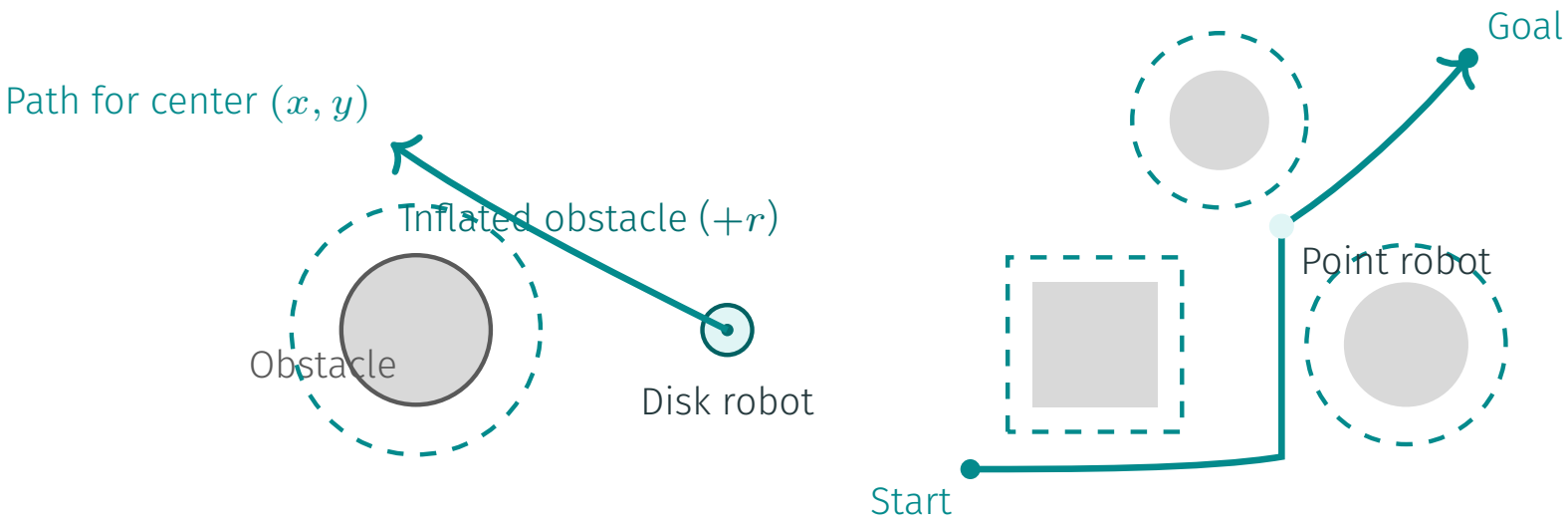
Motion Planning for a Disk Robot

Idea: Represent the disk robot as a **point** in a modified map where obstacles are **inflated** by the robot's radius r.

- Robot: a disk of radius r, configuration (x, y).
- Inflate each obstacle $\mathcal{O}$ by r:

$$\mathcal{C}_{obs} = \{(x, y) \mid \text{distance}((x, y), \mathcal{O}) \leq r\}$$

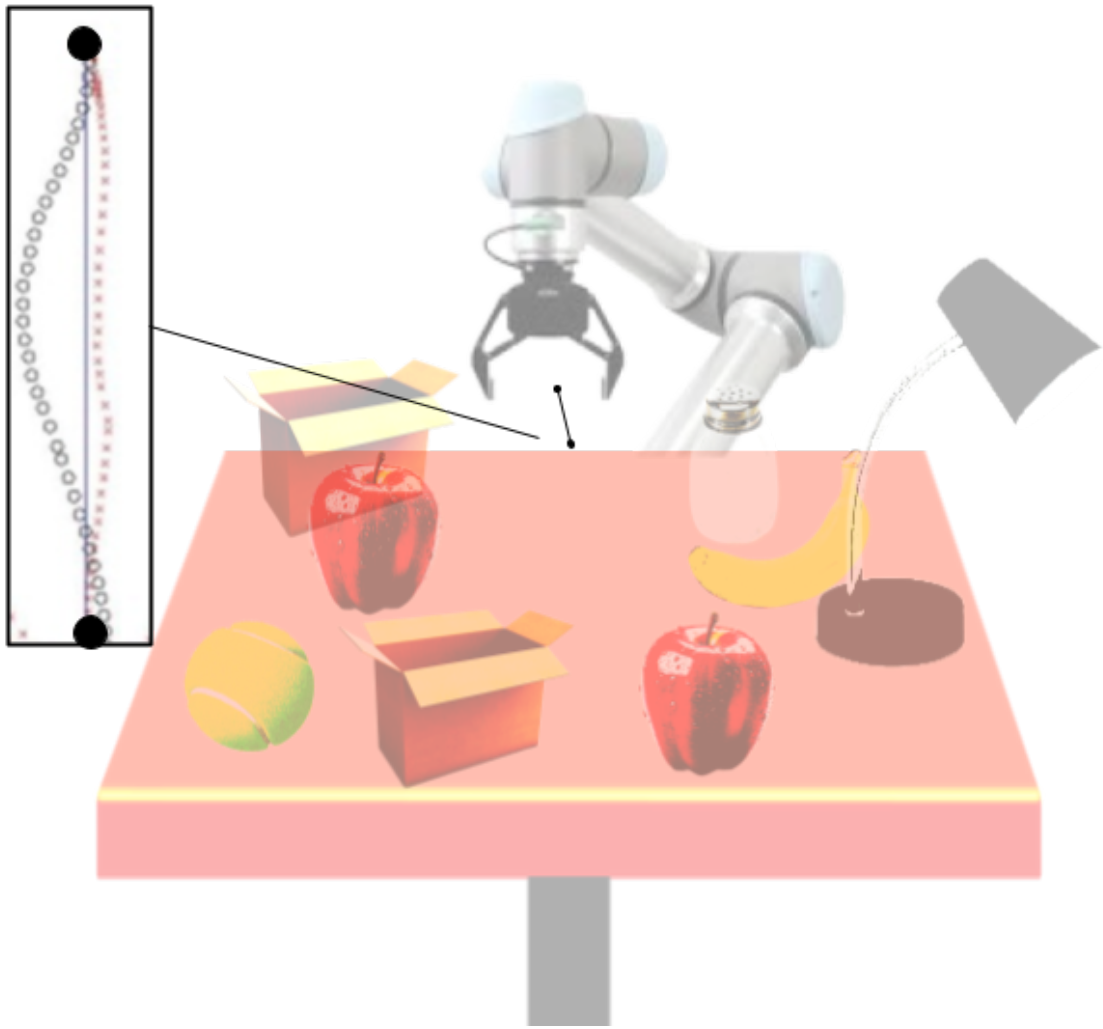
- Plan for a **point robot** in $\mathcal{C}_{free} = \mathbb{R}^2 \setminus \mathcal{C}_{obs}$.



Planning for the **center point** in the inflated space guarantees collision-free motion of the disk robot.

Control

Motion Control

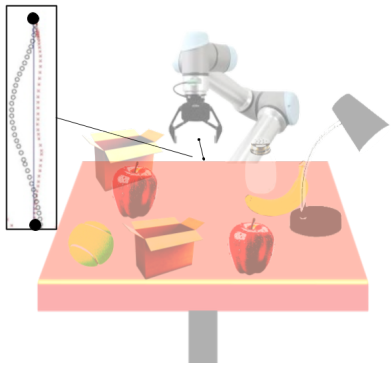


Layered Decision-Making
Task Planning
Motion Planning
Control

Layered Decision-Making
Task Planning
Motion Planning
Control

Motion Control

Given a desired trajectory $\tau(t)$ for a robotic system $\mathcal{R}$ with d **degrees of freedom**, the goal of motion control is to compute and apply control inputs $\mathbf{u}(t)$ (e.g., joint torques, velocities, or forces) such that the system's actual motion $q(t)$ follows $\tau(t)$ as closely as possible.



Control Approaches

Translate a desired behavior into actions using sensor feedback.

- **Rule-based control:** select actions using fixed rules.
  Example: too close to the wall ⇒ turn away.
- **Proportional (P) control:** react to the current error.

$$u(t) = K_P e(t)$$

- **PID control:** combine current error, accumulated error, and the rate of change of the error.

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau)\, d\tau + K_D \frac{de(t)}{dt}$$

  Integral action reduces persistent offset; derivative action can damp oscillations.

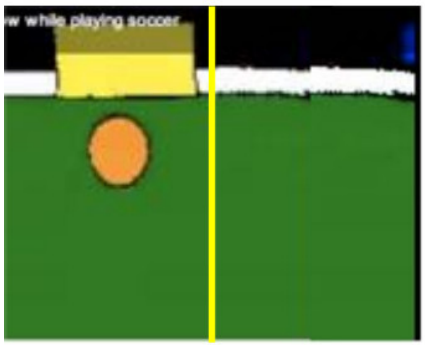
- **Model Predictive Control (MPC):** use a dynamics model to optimize a sequence of actions, execute the first, and replan using new feedback.

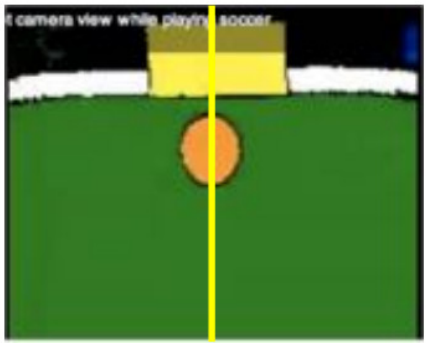
Layered Decision-Making
Task Planning
Motion Planning
Control

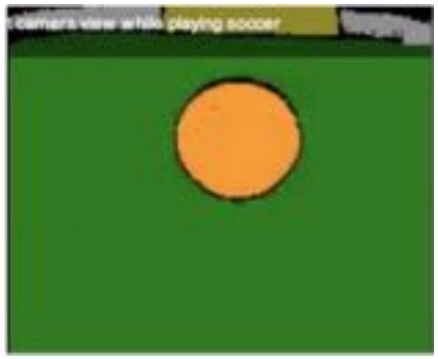
Reactive navigation with visual homing

Purely reactive navigation

- Measure visual (x,y) position of Goal
- Move to bring the goal to the visual center
- Proportional control (if you see the goal), random walk (if you don't)

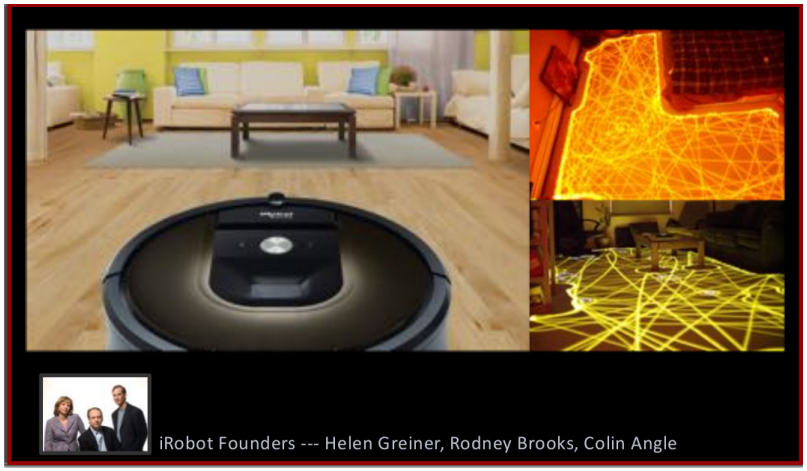


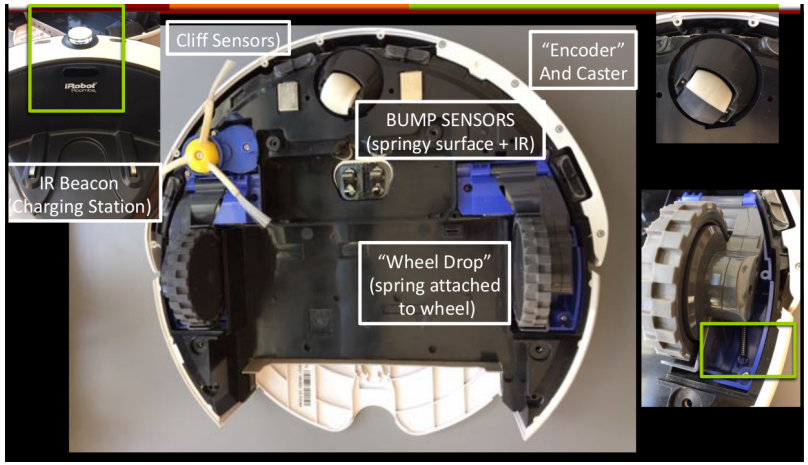




https://www.youtube.com/watch?v=so9axknlftk

Roomba





SIMPLE ROOMBA

If BUMP = TRUE
THEN Turn (random direction/amt)

ELSE MOVE-STRAIGHT

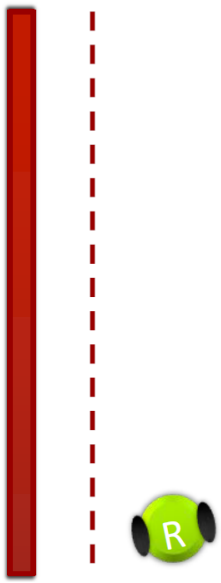
https://www.youtube.com/watch?v=7FSUtSurqA4

- A simple roomba does a random walk to cover a complex unknown space.
- A more complex roomba, might have more bumpers (left/right) or might have more interesting default behaviors (zigzag, spirals). But its still pretty simple.....

Layered Decision-Making
Task Planning
Motion Planning
Control

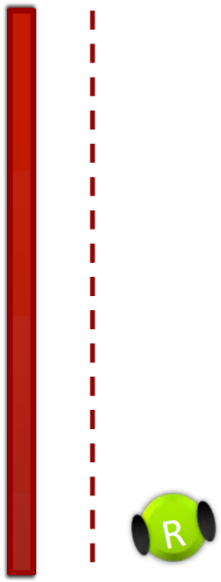
Example

Layered Decision-Making
Task Planning
Motion Planning
Control



Simple scenario: **wall following robot**
Trying to move along an infinite straight wall while maintaining a fixed distance.

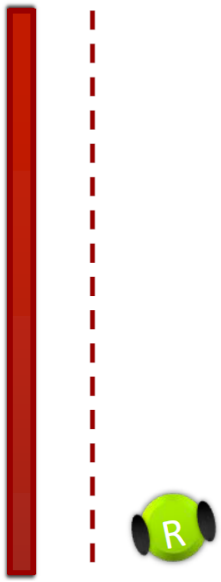
Example - wall following robot



Generic Program Loop

```
Move 1 step forward
If distance-to-wall > desired,
    Then turn towards the wall
    Else turn away from the wall
```

Example - wall following robot



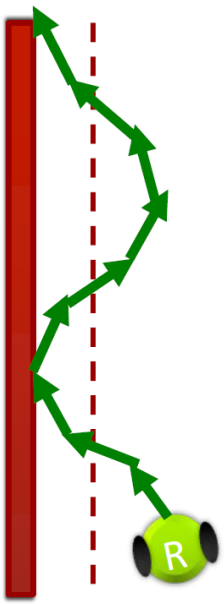
Generic Program Loop

```
Move 1 step forward
If distance-to-wall > desired,
    Then turn towards the wall
    Else turn away from the wall
```

What is a concrete implementation of this generic program?

Example - wall following robot

Concrete implementation

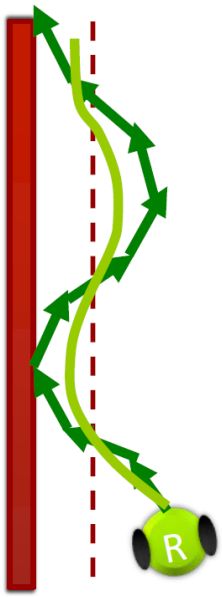


Concrete Program Loop

```
Move 0.5 body-length forward
If distance-to-wall > desired,
    Then turn 45 degrees towards the wall
    Else turn 45 degrees away from the wall
```

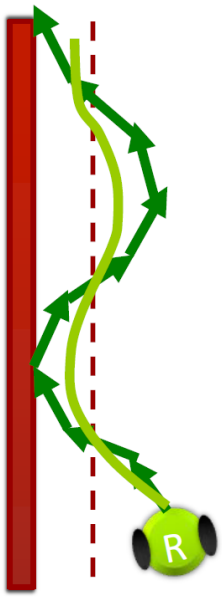
Can we do better?

Proportional (P) control



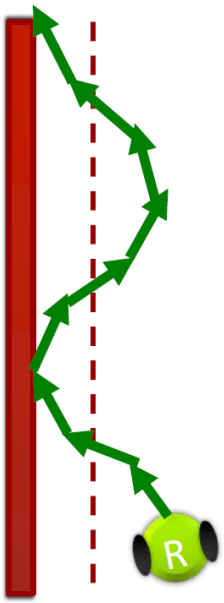
The correction applied to the controlled variable is **proportional** to the difference between the desired value and the measured value

Proportional (P) control



- High-level idea: adjust proportionally to the error
  - If 'far' from the line — turn sharply
  - If 'close' to the line — turn very slowly
- Decide on how much to adjust based on model or experiments.

Proportional (P) control



Proportional Control Program Loop

```
Move 0.5 body-length forward
If distance-to-wall > desired,
    let error = |desired – distance-to-wall|
    Then turn Kp * error towards the wall
    Else turn  Kp * error away from the wall
```

Can we do better?

Layered Decision-Making
Task Planning
Motion Planning
Control

Model Predictive Control (MPC)

Look ahead using a model; act using feedback.

1. **Observe:** estimate the robot's current state.
2. **Predict:** simulate future motion under candidate sequences of control inputs.
3. **Optimize:** choose a sequence that minimizes tracking error and control effort, subject to constraints.
4. **Execute:** apply only the first control input, then observe and repeat.

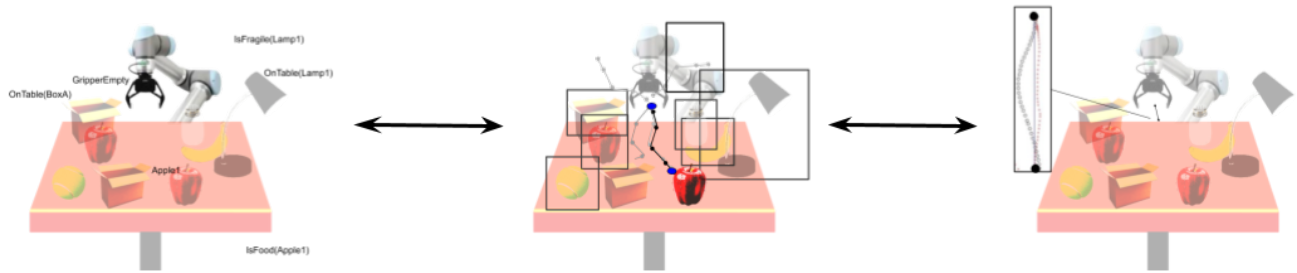
Wall-following example:

- Predict future distance and orientation relative to the wall.
- Choose steering inputs to maintain the desired distance, avoid collisions, and limit abrupt turns.

Trade-off: anticipates future consequences and handles constraints, but

Open Questions

- How to update the state representation?
- How to interface the three layers?
  - For example, which component maintains valid grasp configurations?
- How to account for uncertainty?



Do we need this decoupling between the layers given the increasingly powerful foundation models?

Layered Decision-Making
Task Planning
Motion Planning
Control